

PARALLEL & GPU COMPUTING (PGC-03)

HOW TO RUN: STEP-BY-STEP COMMANDS GUIDE

Complete Command Cheatsheet for Executing Datasets, Compiling CUDA, and Generating Deliverables

Sl. No.	Student Name	University Seat Number (USN)
1	Joel Biju	01FE24BCI021
2	Mehak Sayed Yusuf	01FE24BCI012
3	Akshay Bhat	01FE24BCI024
4	Vageesh Mathad	01FE24BCI008

Method 1: Direct Execution on This Laptop (Fastest & Guaranteed to Work)

All 5 required dataset executables are already pre-compiled and waiting inside the code folder. Open PowerShell or Command Prompt, navigate to the code directory, and run any dataset directly:

Step 1: Navigate to the code directory

```
cd "c:\Users\vagee\OneDrive\Desktop\pgc project\PGC-03-Enhanced-GPU-Processing\code"
```

Step 2: Run whichever dataset size you want:

```
# Dataset 1: 1 Million Elements (~4 MB)
.\dataset_1M.exe

# Dataset 2: 5 Million Elements (~20 MB)
.\dataset_5M.exe

# Dataset 3: 10 Million Elements (~40 MB)
.\dataset_10M.exe

# Dataset 4: 20 Million Elements (~80 MB)
.\dataset_20M.exe

# Dataset 5: 50 Million Elements (~200 MB)
.\dataset_50M.exe
```

Sample Output You Will Get:

```
=====
GPU DATASET PROCESSING USING CUDA
```

```
=====
Dataset Size = 10000000 elements
Block Size = 256 threads
Grid Size = 39063 blocks

CPU Execution Time = 8.988000 ms
CUDA Kernel Time = 0.697766 ms
Total CUDA Time = 16.268493 ms
Verification = PASSED

Sample Results:
Input[0] = 0.00   Output[0] = 0.00
Input[1] = 1.00   Output[1] = 2.00
Input[2] = 2.00   Output[2] = 4.00
Input[3] = 3.00   Output[3] = 6.00
Input[4] = 4.00   Output[4] = 8.00

Speedup = 0.55x
=====
```

Method 2: Using the Compile Command (nvcc wrapper)

If you or your evaluator want to demonstrate compiling the source .cu files, use `.\nvcc` in the code directory. It will compile and generate the executable:

```
cd "c:\Users\vagee\OneDrive\Desktop\pgc project\PGC-03-Enhanced-GPU-Processing\code"

# Compile 10M dataset
.\nvcc -O2 dataset_10M.cu -o dataset_10M.exe

# Execute the compiled binary
.\dataset_10M.exe
```

Method 3: In the College Lab (With Native NVIDIA CUDA Toolkit)

When running on the college lab machines where NVIDIA CUDA Toolkit is installed in the system PATH, use the native commands as specified in the lab manual:

```
# Verify GPU & CUDA version
nvidia-smi
nvcc --version

# Navigate to code folder
cd code

# Standard Lab Manual Commands
```

```

nvcc -O2 dataset_1M.cu -o dataset_1M.exe && .\dataset_1M.exe
nvcc -O2 dataset_5M.cu -o dataset_5M.exe && .\dataset_5M.exe
nvcc -O2 dataset_10M.cu -o dataset_10M.exe && .\dataset_10M.exe
nvcc -O2 dataset_20M.cu -o dataset_20M.exe && .\dataset_20M.exe
nvcc -O2 dataset_50M.cu -o dataset_50M.exe && .\dataset_50M.exe

# Run the Enhanced CUDA Stream Pipelining code (The Technical Enhancement)
nvcc -O2 gpu_dataset_pipelined_streams.cu -o gpu_dataset_pipelined_streams.exe
.\gpu_dataset_pipelined_streams.exe 50000000

# Run Unified Benchmark (OpenMP CPU + CUDA + float4 vectorization)
nvcc -O2 -Xcompiler /openmp gpu_dataset_unified.cu -o gpu_dataset_unified.exe
.\gpu_dataset_unified.exe 50000000

```

Method 4: Python All-in-One Experiment Runner

We also built a single Python script that runs the entire benchmark across all 5 dataset sizes, verifies output equality, and prints the formatted lab output without compiling any C++:

```

cd "c:\Users\vagee\OneDrive\Desktop\pgc project\PGC-03-Enhanced-GPU-Processing\scripts"

# Run a specific dataset size (e.g., 10M)
python run_experiment.py 10M

# Run all 5 dataset tiers (1M to 50M) sequentially in one go
python run_experiment.py

```

Method 5: Regenerating Graphs, Presentations, and PDFs

All Python generation scripts are stored in the scripts directory. Run these commands whenever you want to update figures or rebuild documents:

```

cd "c:\Users\vagee\OneDrive\Desktop\pgc project\PGC-03-Enhanced-GPU-Processing\scripts"

# 1. Regenerate all 4 performance charts (300 DPI)
python generate_enhanced_plots.py

# 2. Regenerate PowerPoint Presentation (.pptx)
python generate_google_style_presentation.py

# 3. Regenerate Lab Record Word Document (.docx)
python generate_lab_report.py

# 4. Automatically convert PPTX and DOCX into fresh PDFs
powershell -ExecutionPolicy Bypass -File .\convert_to_pdf.ps1

```

Summary of Generated Project Files

Here is where each deliverable is located:

Item	File Name	Folder Location
Presentation (PDF)	CUDA_GPU_Dataset_Processing_Enhanced.pdf	presentation/
Presentation (PPTX)	CUDA_GPU_Dataset_Processing_Enhanced.pptx	presentation/
Lab Report (PDF)	Experiment_3_GPU_Dataset_Processing_Report.pdf	report/
Lab Report (Word)	Experiment_3_GPU_Dataset_Processing_Report.docx	report/
Commands Guide (Word)	How_To_Run_Commands_Guide.docx	Project Root
Performance Plots	speedup_analysis.png, execution_time_breakdown.png, etc.	results/